MP6: Primitive Device Driver

Mohammed Sharique
UIN: 837004870
CSCE611: Operating System

04/12/2026

Assigned Tasks

Main: Completed.
Bonus 1: Completed.
Bonus 2: Completed.
Bonus 3: Not Completed.

System Design

The goal of this machine problem was to improve the disk device driver so that it no longer ties up the CPU while waiting for the physical disk to become ready during I/O operations. The main idea was to implement a `NonBlockingDisk` device built upon the foundation of the low-level `SimpleDisk`. By utilizing the thread scheduler and an interrupt-driven architecture, the heavily inefficient busy-waiting loop was eliminated, allowing the operating system to successfully execute other threads concurrently while the disk controller fetches or writes data.

- I first designed the basic `NonBlockingDisk` class by deriving it from both the existing `SimpleDisk` hardware controller and the `InterruptHandler` to securely integrate it alongside the native IDE IRQ 14 pipeline.
- I then eliminated the primary polling bottleneck by overriding the `wait_while_busy()` function. Instead of spinning aimlessly in a hardware loop, the thread requesting disk access actively yields control over the processor back to the `Scheduler`.
- To maintain robust thread-safety and prevent race conditions from concurrent I/O requests, I utilized a Split-Driver architecture mapping the hardware reads and writes natively through a centralized `disk_queue` array of capacity 8 that functions identical to the kernel's scheduler ready queue.
- I then shifted the execution to properly manage synchronous top-half and asynchronous bottom-half logic. The `NonBlockingDisk::read` and `write` functions generate requests natively inside a `Machine::disable_interrupts()` locking boundary preventing parallel corruption by multiple threads communicating with the hardware IDE ports.
- Lastly, I implemented the asynchronous interrupt handler driven directly by the disk controller. Rather than constantly checking status registers, the `handle_interrupt()` method catches the IRQ 14 completion signal, executes the hardware data fetches directly through the bottom-half, and automatically resumes the specifically queued thread safely back into the scheduler rotation to continue its task.

Bonus 1: Concurrency & Race Conditions in Disk I/O

When translating a simple polling-based hardware model into a preemptively multi-threaded kernel architecture, guaranteeing thread-safe concurrency for shared physical resources becomes paramount.

The Primary IDE Controller only possesses a solitary set of low-level hardware communication ports (`0x1F0 - 0x1F7`). Because multiple distinct threads independently attempting to read or write disk blocks concurrently are functionally blind to one another, several major race conditions naturally arise:

- **Hardware Port Corruption & Preemption:** If Thread A calls `read()` and starts issuing the sector target configurations sequentially to the disk hardware (LBA ports `0x1F3-0x1F5`), but its quantum expires and the timer forcefully preempts it midway through, Thread B may step in. If Thread B calls `write()`, it will recklessly overwrite those same LBA ports. When Thread A eventually resumes, it sends its final read command—resulting in both operations targeting entirely mangled sectors, causing catastrophic data corruption across the disk platter.
- **Interrupt Wakeup Hijacking:** While executing `wait_while_busy()`, a thread actively bounds itself by yielding until it catches `IRQ 14`. If two concurrent threads attempt an unbounded operation simultaneously without queuing, the active `current_disk_thread` pointer will be simply overwritten. When the disk interrupt finally arrives, only the latter thread awakens, leaving the first thread asleep permanently inside an unrecoverable deadlock.

Safe Concurrent Handling Strategy

To natively solve this problem while maintaining the fundamental interfaces of the `SimpleDisk` class, robust mutual exclusion structures were designed squarely inside the `NonBlockingDisk` wrapper.

- **Critical Section Interlocking:** A primitive mutex equivalent, `is_disk_in_use`, safely manages access to the controller. To protect this boolean flag from preemptive race conditions itself, it is only exclusively evaluated and modified within a `Machine::disable_interrupts()` locking boundary. This ensures complete atomicity by disabling the system timer, effectively preventing any thread context swaps until the lock state resolves cleanly.
- **Asynchronous Thread Queuing:** Threads encountering an actively occupied hardware controller are cleanly serialized onto `disk_queue`, an array functioning explicitly like a blocked-scheduler. Instead of locking the CPU in an endless polling loop when the disk is busy, the calling framework forces the thread to push its configuration sequentially to the queue boundaries, re-enables external interrupts, and systematically yields execution. Once a hardware operation completes natively causing an interrupt, `handle_interrupt` selectively revives the exact next thread from this array using `SCHEDULER->resume()`, granting it exclusive authority over the disk hardware without risking any collateral concurrent overlaps.

Code Description

I modified the files nonblocking_disk.C/H and kernel.C (just to enable execution of nonblocking disk) and brought these files in from MP5 scheduler.C/H, threads.C/H, interrupts.C.
The code compiles by running make in the project directory.
Github: https://github.com/shariqu3/csce-611-machine-problems/tree/v2

nonblocking_disk.C:NonBlockingDisk : This constructor initializes the non-blocking disk subsystem layered on top of its inherited `SimpleDisk` interface. It establishes the foundational thread-safe architecture by explicitly zeroing out the `n_threads` tracking variable, clearing the initial sequence parameters of the bounded `disk_queue` array buffers, setting the internal tracking pointer `current_disk_thread` to null, and freeing the `is_disk_in_use` mutex flag. Finally, it inextricably links the device into the kernel's native interrupt topology by definitively registering the class instance as the active handler for `IRQ 14` using `InterruptHandler::register_handler()`.

nonblocking_disk.C:wait_while_busy : This function overrides the default polling loop in `SimpleDisk`. Instead of continuously checking hardware status using a busy loop, it captures the currently active dispatching thread into `current_disk_thread` and actively calls `System::SCHEDULER->yield()`. This yields CPU execution smoothly back to the scheduler, suspending the waiting thread until the hardware signals completion natively through an interrupt.

```
NonBlockingDisk::NonBlockingDisk(unsigned int _size)
  : SimpleDisk(_size) {
    start = 0;
    n_threads = 0;
    is_disk_in_use = false;
    current_disk_thread = nullptr;
    for (int i = 0; i < 8; i++) {
        disk_queue[i] = nullptr;
    }
    InterruptHandler::register_handler(14, this);
}
```

```
void NonBlockingDisk::wait_while_busy() {
    bool is_ints_enabled = Machine::interrupts_enabled();
    Machine::disable_interrupts();
    while (is_busy()) {
        current_disk_thread = Thread::CurrentThread();
        if (is_ints_enabled) Machine::enable_interrupts();
        System::SCHEDULER->yield();
        Machine::disable_interrupts();
    }
    if (is_ints_enabled) Machine::enable_interrupts();
}
```

nonblocking_disk.C:read : This function operates as the thread-safe top-half for disk read operations. It first disables interrupts to create a critical section, protecting the `is_disk_in_use` mutex mechanism. If the disk is currently occupied, it pushes the calling thread onto a circular `disk_queue`, restores interrupts, and yields control so the thread sleeps. Once the hardware is available, it locks the mutex, initiates the `SimpleDisk::read` sequence, and upon completion securely pops the next waiting thread from the queue returning it using `System::SCHEDULER->resume()`.

nonblocking_disk.C:write : Symmetrical to the read function, this handles synchronous sector writes. It actively protects hardware port registers by tracking the `is_disk_in_use` flag exactly alongside the `disk_queue` bounded array. Threads requesting writes during an ongoing operation are securely appended sequentially and yielded natively. Once execution completes successfully, the mutex releases and the next sleeping thread is cleanly scheduled.

nonblocking_disk.C:handle_interrupt : This function handles the fundamental asynchronous bottom-half logic executed directly upon receiving `IRQ 14` from the Primary IDE Controller. Serving as the wakeup mechanism for the suspended `wait_while_busy` sequence, it captures the `current_disk_thread` reference safely, and dispatches it directly back into processor execution sequentially utilizing `System::SCHEDULER->resume` ensuring that hardware interruptions efficiently revive threads originally blocked by operations.

Testing

For this machine problem, I relied on the provided kernel test loads fun1 through fun4 for validating the implementation. The test setup aggressively exercised concurrent disk access, thread yielding, and asynchronous interrupt handling while observing serial output for scheduling order, successful sector reads and writes, and overall system stability.

Initially, I encountered issues regarding race conditions when multiple threads attempted to evaluate the SimpleDisk hardware concurrently, resulting in sector target corruption and missed IRQ 14 completion interrupts. This was resolved by designing the is_disk_in_use mutex boundary, protecting the

```
void NonBlockingDisk::read(unsigned long _block_no, unsigned char* _buf) {
    bool is_ints_enabled = Machine::interrupts_enabled();
    if (is_ints_enabled) Machine::disable_interrupts();

    while (is_disk_in_use) {
        if (n_threads < 8) {
            disk_queue[(start + n_threads++) % 8] = Thread::CurrentThread();
        } else {
            Console::puts("Disk queue full!\n");
            assert(false);
        }

        if (is_ints_enabled) Machine::enable_interrupts();
        System::SCHEDULER->yield(); // Wait to be dequeued
        if (is_ints_enabled) Machine::disable_interrupts();
    }

    is_disk_in_use = true;
    if (is_ints_enabled) Machine::enable_interrupts();

    // Actual operation
    SimpleDisk::read(_block_no, _buf);

    if (is_ints_enabled) Machine::disable_interrupts();
    is_disk_in_use = false;

    if (n_threads > 0) {
        Thread* next = disk_queue[start];
        disk_queue[start] = nullptr;
        start = (start + 1) % 8;
        n_threads--;
        System::SCHEDULER->resume(next);
    }
    if (is_ints_enabled) Machine::enable_interrupts();
}
```

disk request phase entirely inside a critical section disabled from hardware interrupts. I also debugged scheduler queueing logic to successfully store and release multiple sleeping disk threads sequentially without bleeding into the generic CPU ready-queue context.

```
void NonBlockingDisk::write(unsigned long _block_no, unsigned char* _buf) {
    bool is_ints_enabled = Machine::interrupts_enabled();
    if (is_ints_enabled) Machine::disable_interrupts();

    while (is_disk_in_use) {
        if (n_threads < 8) {
            disk_queue[(start + n_threads++) % 8] = Thread::CurrentThread();
        } else {
            Console::puts("Disk queue full!\n");
            assert(false);
        }

        if (is_ints_enabled) Machine::enable_interrupts();
        System::SCHEDULER->yield(); // Wait to be dequeued
        if (is_ints_enabled) Machine::disable_interrupts();
    }

    is_disk_in_use = true;
    if (is_ints_enabled) Machine::enable_interrupts();

    // Actual operation
    SimpleDisk::write(_block_no, _buf);

    if (is_ints_enabled) Machine::disable_interrupts();
    is_disk_in_use = false;

    if (n_threads > 0) {
        Thread* next = disk_queue[start];
        disk_queue[start] = nullptr;
        start = (start + 1) % 8;
        n_threads--;
        System::SCHEDULER->resume(next);
    }
    if (is_ints_enabled) Machine::enable_interrupts();
}
```

```
void NonBlockingDisk::handle_interrupt(REGS * _r) {
    if (current_disk_thread != nullptr) {
        Thread* t = current_disk_thread;
        current_disk_thread = nullptr;
        System::SCHEDULER->resume(t);
    }
}
```